



EE200

Signals, Systems & Networks

PRACTICAL REPORT

Name: SHUBHAM JOSHI

Roll no : 240500

Instructor: Dr. TUSHAR SANDHAN

Date of Submission: 05-07-2025



TABLE OF CONTENTS

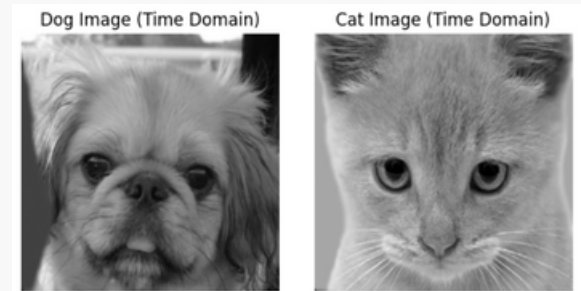
	page no.
1. Frequency mixer: 'Beauty and the Blur'	1-3
1.1 Initial Setup: Tools and Input Images	1
1.2 Fourier Transform, Spectral Analysis, and Rotation Effects	1
1.3 Image Fusion and Reconstruction using Masked Spectra	2
1.4 About Transfer Function	3
2. Frequency de-mixer: 'Unwanted Solo'	4-8
1.1 Audio Loading and Preprocessing	4
1.2 . Frequency Domain Analysis Using FFT and PSD	4
1.3 Observations and Justification for Frequency Suppression	5
1.4 Peak Frequency Detection	6
1.5 Filter Design: Bandstop Filtering	6
1.6 Noise Removal by Filtering	7
1.8 Exporting the Cleaned Audio	8

1. Frequency mixer: 'Beauty and the Blur'

1.1 Initial Setup: Tools and Input Images

The required libraries were imported, including:

1. PIL for image loading
2. NumPy for numerical operations
3. matplotlib for visualization
4. numpy.fft for Fourier transform computations.



Two grayscale images, namely *dog_gray.jpg* and *cat_gray.jpg*, were selected for processing. They were resized to a resolution of 512×512 pixels and converted to 8-bit grayscale format.

1.2 Fourier Transform, Spectral Analysis, and Rotation Effects

To analyze the frequency characteristics of the input image, the 2D Discrete Fourier Transform (DFT) was applied using the *fft2()* function. This transformation converted the image from the spatial domain into the frequency domain, where each pixel represents a specific spatial frequency component.

The magnitude spectrum was calculated using the absolute value of the complex FFT output. Additionally, a logarithmic magnitude spectrum (in decibels) was computed to enhance low-energy components:

$$|F(u, v)|_{dB} = 20 \cdot \log_{10}(|F(u, v)| + \epsilon), \quad \epsilon = 10^{-5}$$

This conversion was performed to enhance the visibility of lower-intensity frequency components. Without this step, the spectrum tends to be dominated by a bright central region corresponding to the low frequencies.

The frequency spectrum was also centered using *fftshift()* to reposition the zero-frequency (DC) component to the center of the display. This shift was important because it allows for more intuitive visualization: low frequencies appear in the center, and high frequencies spread outward in all directions.

Rotational Effects in Frequency Domain

To explore how geometric changes affect spectral content, the image was rotated by both 90° and 45°, and the corresponding spectra were recalculated. This step was crucial to confirm a key theoretical property of the Fourier Transform:

A rotation in the spatial domain results in an identical rotation in the frequency domain.

Image rotated by 90°

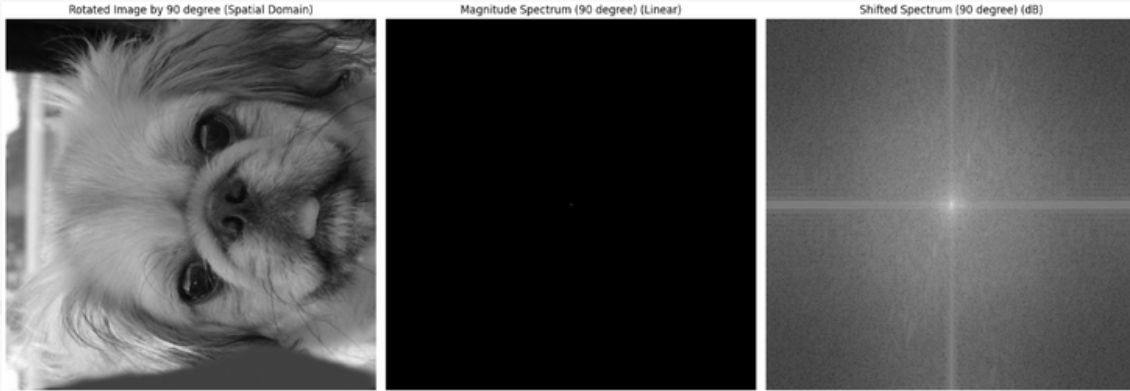
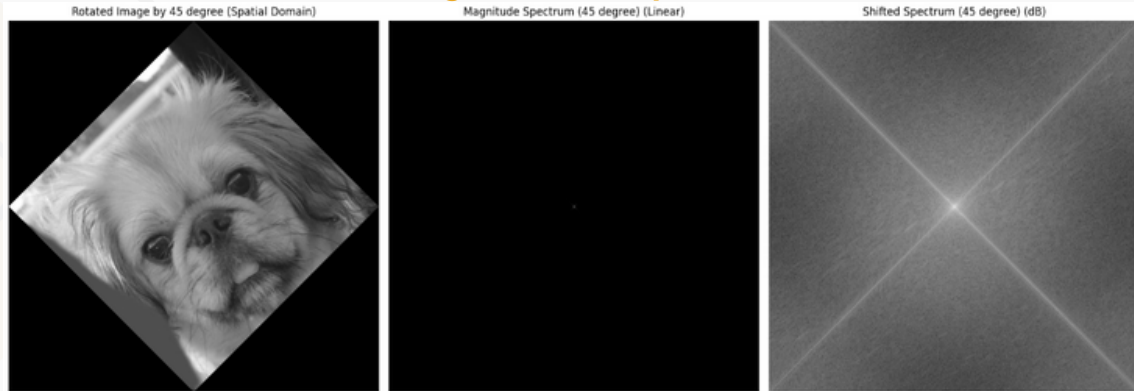


Image rotated by 45°



It was observed that the frequency spectrum rotated in the same direction and by the same angle as the spatial image. This validated the theoretical property of the Fourier Transform:

'A rotation in the spatial domain results in an equivalent rotation in the frequency domain.'

This property confirms that the geometry and orientation of the image are preserved in its frequency representation — a critical insight before proceeding to frequency-based fusion.

1.3 Image Fusion and Reconstruction using Masked Spectra

To achieve frequency-based image fusion, the magnitude and phase spectra of both input images were extracted from their respective Fourier Transforms. The fusion was done in such a way that low-frequency components (structure) were taken from one image, and high-frequency components (details) were taken from the other.

Step 1: Extraction of Spectral Components

For each Image:

- The 2D FFT was computed:
- Magnitude and phase were extracted: $|F_i(u, v)| = \text{Magnitude}$, $\angle F_i(u, v) = \text{Phase}$, $i \in \{1, 2\}$

$$F_1(u, v) = \mathcal{F}[I_1(x, y)], \quad F_2(u, v) = \mathcal{F}[I_2(x, y)]$$

Step 2: Designing Frequency Masks

To blend the spectral components smoothly:

- A radial distance matrix was created:
- A low-pass transfer function was applied using an exponential (Gaussian) mask:

$$H_{LP}(u, v) = \exp\left(-\frac{r^4}{2\sigma^4}\right)$$

$$H_{HP}(u, v) = 1 - H_{LP}(u, v) \quad r(u, v) = \sqrt{(u - u_0)^2 + (v - v_0)^2}$$

Step 3: Spectral Fusion

- Using the above masks, the fused magnitude and phase were computed as:

$$|F_{\text{fused}}| = |F_1| \cdot H_{LP} + |F_2| \cdot H_{HP}$$

$$\angle F_{\text{fused}} = \angle F_1 \cdot H_{LP} + \angle F_2 \cdot H_{HP}$$

- They were combined into the complex spectrum:

$$F_{\text{fused}} = |F_{\text{fused}}| \cdot e^{j \cdot \angle F_{\text{fused}}}$$

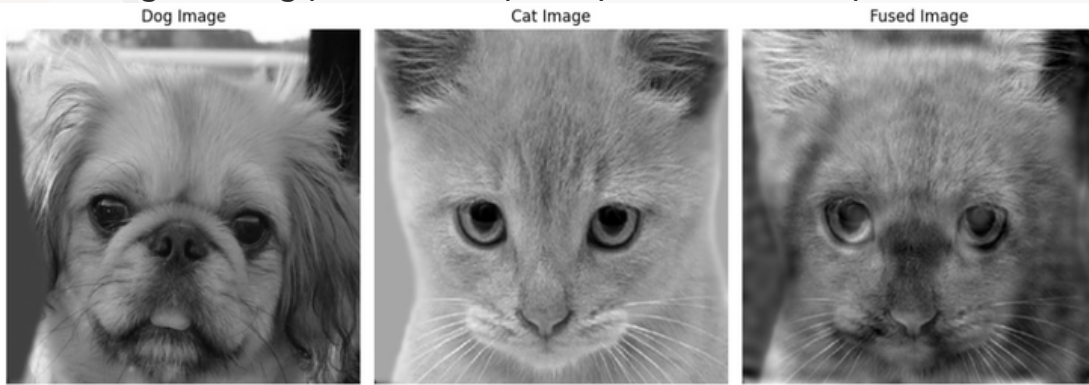
Step 4: Inverse Transform and Final Image

- The spectrum was shifted back using *ifftshift()*
- The inverse FFT was applied:

$$I_{\text{fused}}(x, y) = |\mathcal{F}^{-1}[F_{\text{fused}}]|$$

The absolute value was taken to extract the real-valued image.

The result was a perceptually hybrid image — blending global structure from cat and local details from dog — using precise frequency-domain manipulation.



1.4 About Transfer Function

A 4th-order exponential (Gaussian-like) function was used to define the low-pass mask:

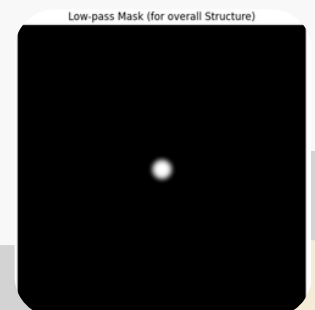
$$H_{LP}(u, v) = \exp\left(-\frac{r^4}{2\sigma^4}\right) \quad \text{where} \quad r = \sqrt{(u - u_0)^2 + (v - v_0)^2}$$

The use of a 4th-order term r^4 (instead of the standard r^2) results in:

- A sharper fall-off near the transition zone
- Better separation of frequency bands
- Reduced spectral leakage or overlap during fusion

Characteristics of the Transfer Function

- Radially symmetric about the DC component
- Smooth, avoiding hard cutoffs (which can introduce ringing artifacts)
- Complementary: $H_{LP}(u, v) + H_{HP}(u, v) = 1 \quad \forall (u, v)$



2. Frequency de-mixer: 'Unwanted Solo'

1.1 Audio Loading and Preprocessing

The analysis begins by importing the audio file using:

```
y, sr = librosa.load(audio_path, sr=None)
```

y is a NumPy array representing the audio waveform, and sr is the sampling rate in Hz. This converts the audio into a numerical format suitable for signal processing.

The sampling rate determines how frequently the signal is sampled per second.

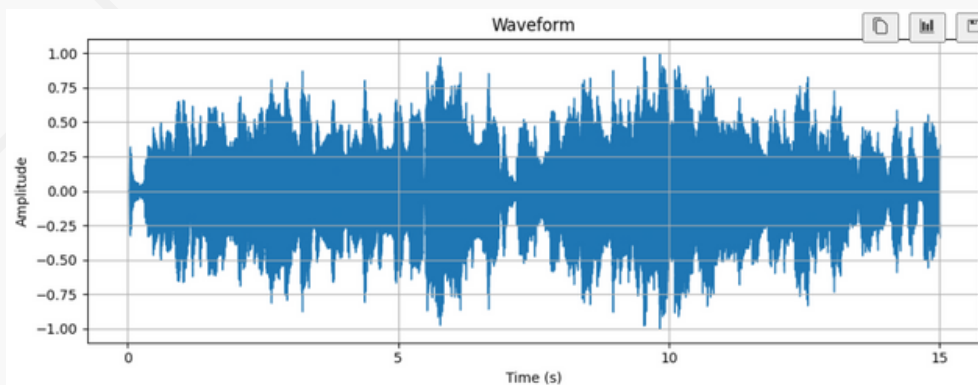
According to **Nyquist's theorem**, to avoid aliasing and accurately capture all frequency components, the sampling rate must satisfy:

$$f_s \geq 2f_{max}$$

then the signal is normalized to prevent clipping and maintain amplitude consistency:

$$y_{normalized} = \frac{y}{\max(|y|)}$$

This scales the signal to the range $[-1, 1]$, which is important for visualization and filtering.



1.2 . Frequency Domain Analysis Using FFT and PSD

To analyze the frequency content of the audio signal, the Fast Fourier Transform (FFT) is applied. FFT is an efficient algorithm that computes the Discrete Fourier Transform (DFT), which converts a signal from the time domain to the frequency domain.

Mathematically, the DFT is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j2\pi kn/N}$$

Where:

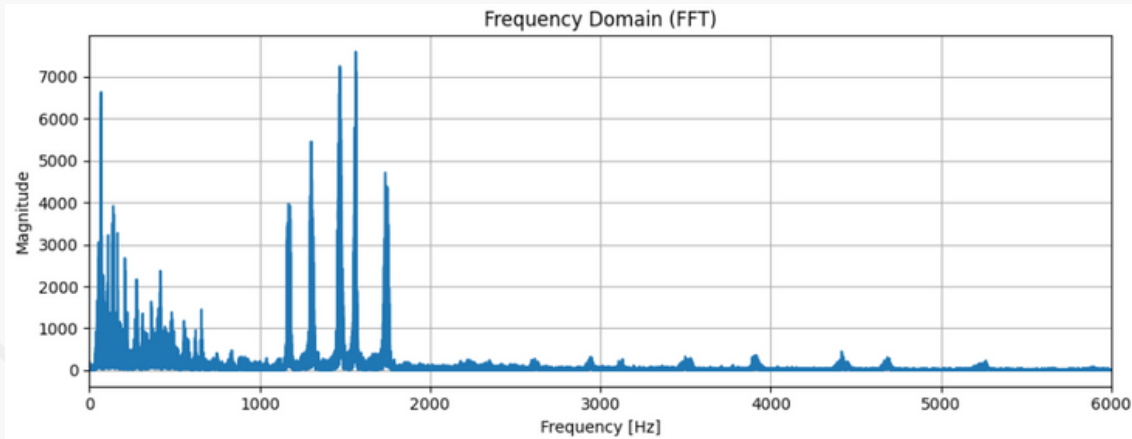
- $x[n]$ is the discrete-time signal
- $X[k]$ is the frequency-domain representation
- N is the total number of samples
- k is the index of the frequency bin

The FFT computes this in $O(N \log N)$ time, making it practical for real-time analysis.

The magnitude & phase spectrum is obtained as:

$$|X[k]| = \sqrt{\text{Re}(X[k])^2 + \text{Im}(X[k])^2}$$

$$\angle X[k] = \arctan\left(\frac{\text{Im}(X[k])}{\text{Re}(X[k])}\right)$$



POWER SPECTRAL DENSITY

While the FFT magnitude spectrum is informative, it is often noisy due to finite sampling and spectral leakage. To obtain a smoother representation, the Power Spectral Density (PSD) is computed using **Welch's method**, implemented in Python as:

```
from scipy.signal import welch  
f, Pxx = welch(y, sr)
```

Where:

- f contains frequency bins
- Pxx contains the estimated power at each frequency

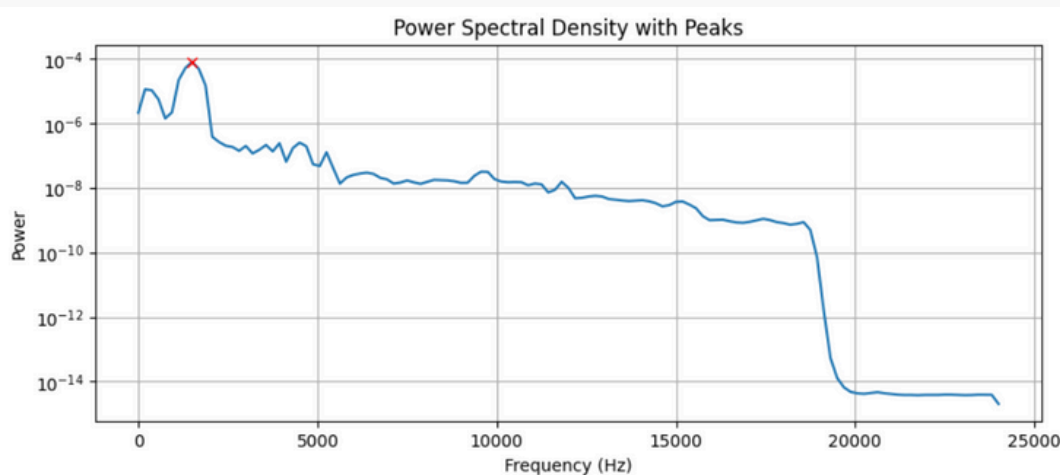
Welch's method works by:

- Dividing the signal into overlapping segments
- Applying a window (e.g. Hamming)
- Computing the FFT of each segment
- Averaging their squared magnitudes

$$PSD(f) = \frac{1}{K} \sum_{k=1}^K |X_k(f)|^2$$

Where:

- K is the number of segments
- $X_k(f)$ is the FFT of the k^{th} windowed segment



Frequency Detected
1500Hz

1.3 Observations and Justification for Frequency Suppression

During the PSD analysis, a strong and narrow spectral peak was observed in the higher frequency region, suggesting the presence of a piccolo flute tone. Given the typical frequency range of the piccolo (~500 Hz to 8000 Hz), it was justified to suppress not only the identified peak but also the broader frequency band from 600 Hz to 8000 Hz.

This decision accounts for the fact that both the **fundamental frequencies** and **harmonic overtones** of the piccolo lie within this range. To achieve this, a **broad bandstop filter** was designed and applied in addition to the **narrow notch filters** targeting specific detected peaks.

1.4 Peak Frequency Detection

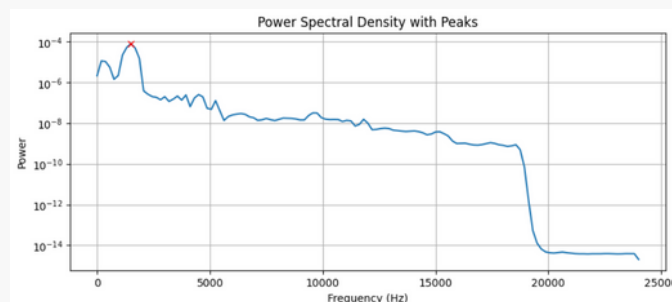
After computing the Power Spectral Density (PSD), the next step is to identify dominant frequencies within the signal — such as interference, harmonics, or instrumental tones like the piccolo flute.

This is achieved using the `scipy.signal.find_peaks()` function:

```
peaks, _ = find_peaks(Pxx, height=threshold, distance=spacing)
```

Where:

- P_{xx} is the PSD array
- `threshold` sets a minimum height (e.g., $0.004 \times \text{max power}$) to isolate significant peaks
- `distance` ensures peaks are spaced apart to avoid false positives



The function detects local maxima in the PSD, corresponding to frequencies with high energy, which often indicate:

- Electrical hums (e.g., 50/60 Hz)
- Instrument tones (e.g., flute or piccolo)
- High-energy harmonics or feedback

In this analysis, a clear and narrow peak was detected in the high-frequency region, corresponding to the piccolo tone embedded in the audio. The frequency of this peak was recorded for the next stage: designing a targeted filter to suppress it.

This automated detection approach ensures:

- High specificity (filters only true peaks)
- Preservation of essential components like speech or music
- Minimal distortion, avoiding unnecessary loss of desired signal content

1.5 Filter Design: Bandstop Filtering

Once the flute's peak frequency is identified, a filter must be designed to suppress only that frequency while preserving the rest of the audio content. A 2nd-order Butterworth **bandstop** filter is selected for its:

- Smooth and monotonic transition characteristics
- Flat passband (minimal distortion of surrounding frequencies)
- Simplicity in both design and implementation

Filter Design Parameters

The normalized cutoff frequencies are computed as:

Where:

- f : the detected flute tone (e.g. ~5000 Hz)
- Δf : the notch bandwidth (e.g., 60 Hz)
- f_s : the sampling rate

$$\text{low} = \frac{f - \Delta f/2}{f_s/2}, \quad \text{high} = \frac{f + \Delta f/2}{f_s/2}$$

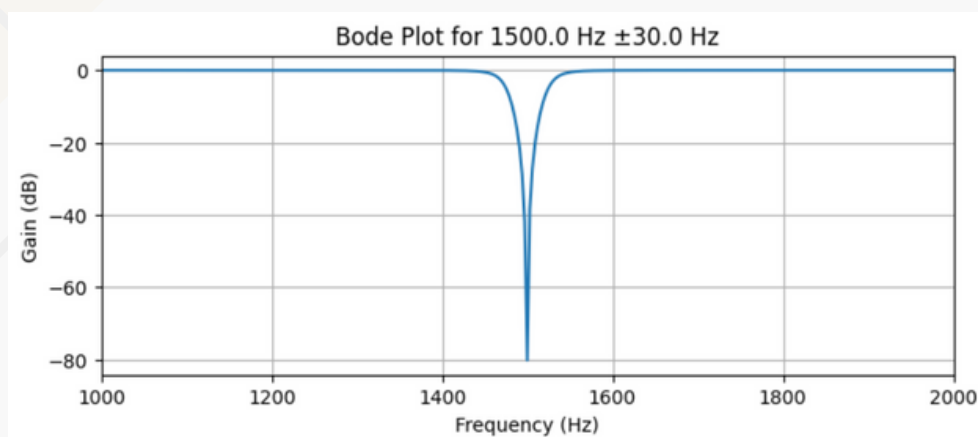
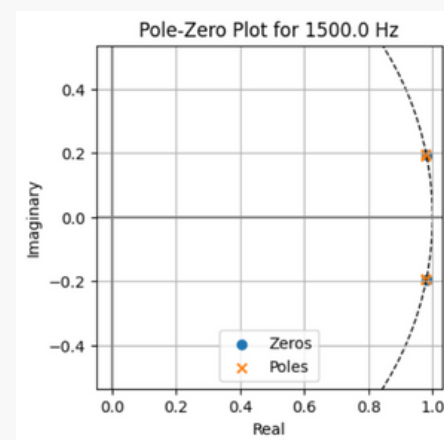
The filter is designed using:

$b, a = \text{butter}(N=2, Wn=[\text{low}, \text{high}], \text{btype}='bandstop')$

This creates a notch filter centered on the flute's frequency, attenuating energy in that narrow band.

Validation

- A Bode plot confirms gain suppression at the target frequency
- A Pole-Zero plot confirms:
 - Zeros lie on the unit circle near the flute frequency → canceling energy
 - Poles are placed just inside the circle → controlling notch sharpness while maintaining stability



1.6 Noise Removal by Filtering

Once the **bandstop filter** is designed, it is applied to the audio signal to suppress the unwanted piccolo tone. To avoid phase distortion, zero-phase filtering is used via the `filtfilt()` function:

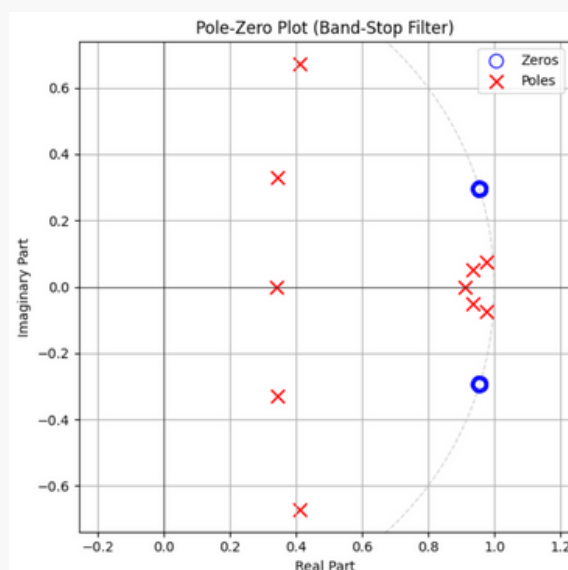
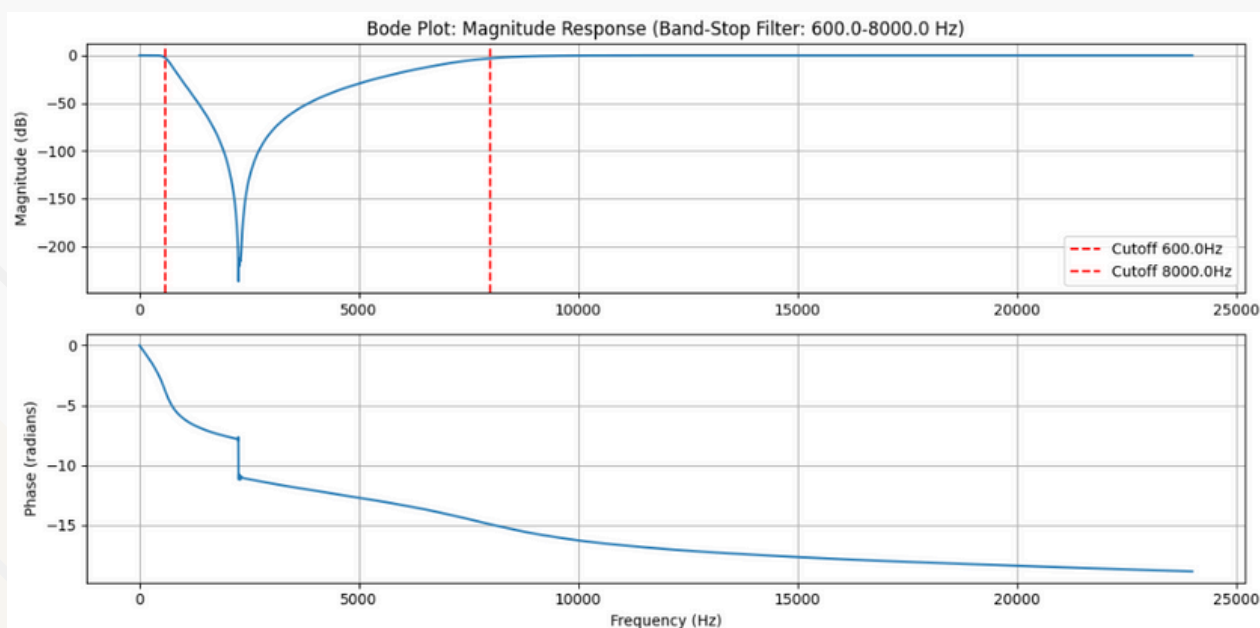
$y_{\text{filtered}} = \text{filtfilt}(b, a, y)$

Where:

- b, a are the filter coefficients from the Butterworth design
- y is the original signal
- y_{filtered} is the output after filtering

Why `filtfilt()` is Used:

- Applies the filter forward and backward, ensuring zero phase distortion
- Maintains timing and waveform shape, which is critical in speech and music
- Minimizes unintended alterations to neighboring frequencies



1.8 Exporting the Cleaned Audio

After filtering, the final processed signal is exported using the **soundfile** library:

```
import soundfile as sf
sf.write("song_piccolo_removed.wav", y_filtered, sr)
```

This saves the filtered signal as a .wav file, preserving the original sampling rate. The resulting file contains the cleaned audio, with the piccolo flute successfully suppressed and all other content, such as vocals and background music, intact.